

1. Установка ОС Linux на виртуальную машину

1.1 Установка VirtualBox

Для организации рабочего места использовалась среда виртуализации Oracle VM VirtualBox, в которой развернута гостевая операционная система Ubuntu Linux. Написание исходного кода выполнялось в текстовом редакторе, а его компиляция и сборка на языке Си осуществлялись с помощью стандартного компилятора GCC через терминал операционной системы. Изображение приветственного окна представлено на рисунке 1.

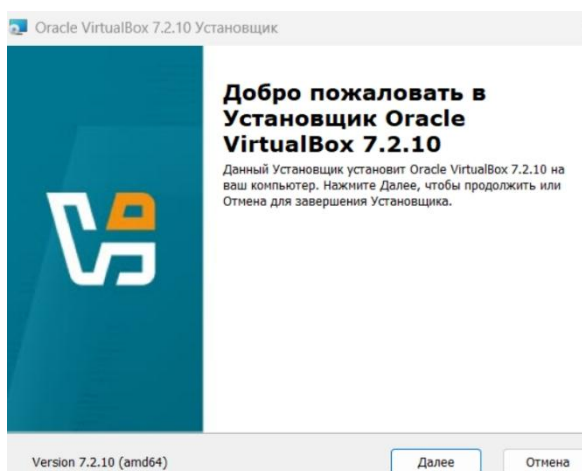


Рисунок 1 – Запуск установочного файла VirtualBox 7.2.10

Далее приняли лицензионное соглашение. Изображение окна представлено на рисунке 2.

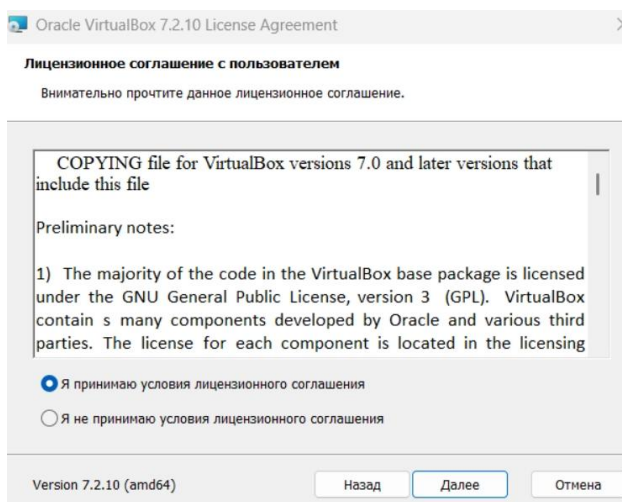


Рисунок 2 – Лицензионное соглашение

Затем выбрали компоненты для установки. Изображение окна представлено на рисунке 3.

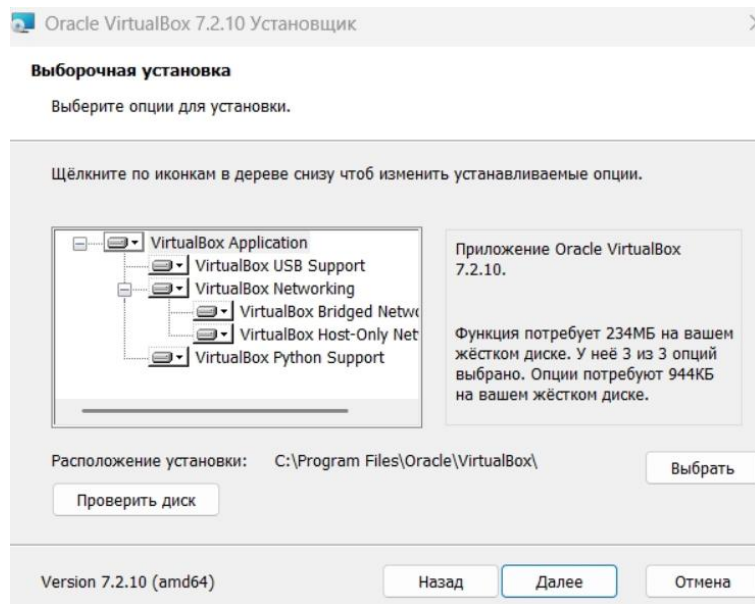


Рисунок 3 – Выбор компонентов для установки

Создали ярлык на рабочем столе. Изображение окна представлено на рисунке 4.

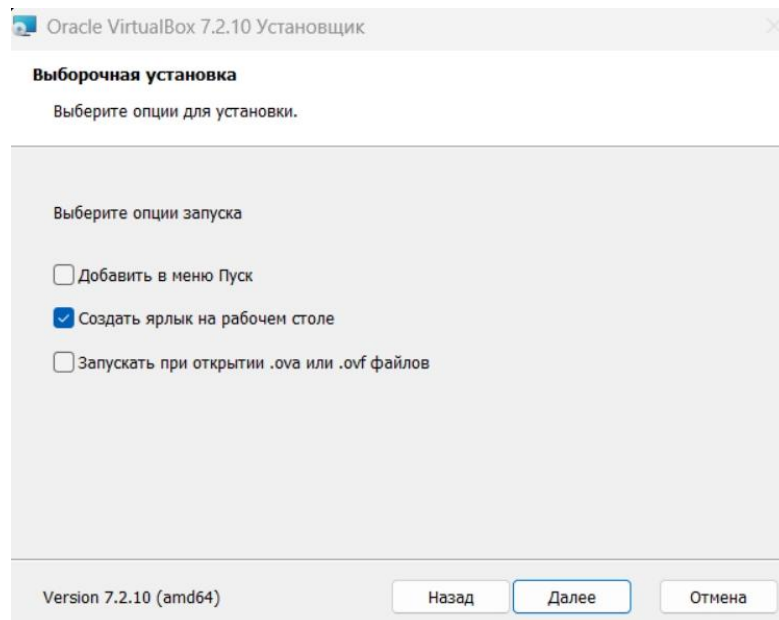


Рисунок 4 – Создание ярлыка на рабочем столе

Открыли Oracle VirtualBox. Изображение окна представлено на рисунке 5.

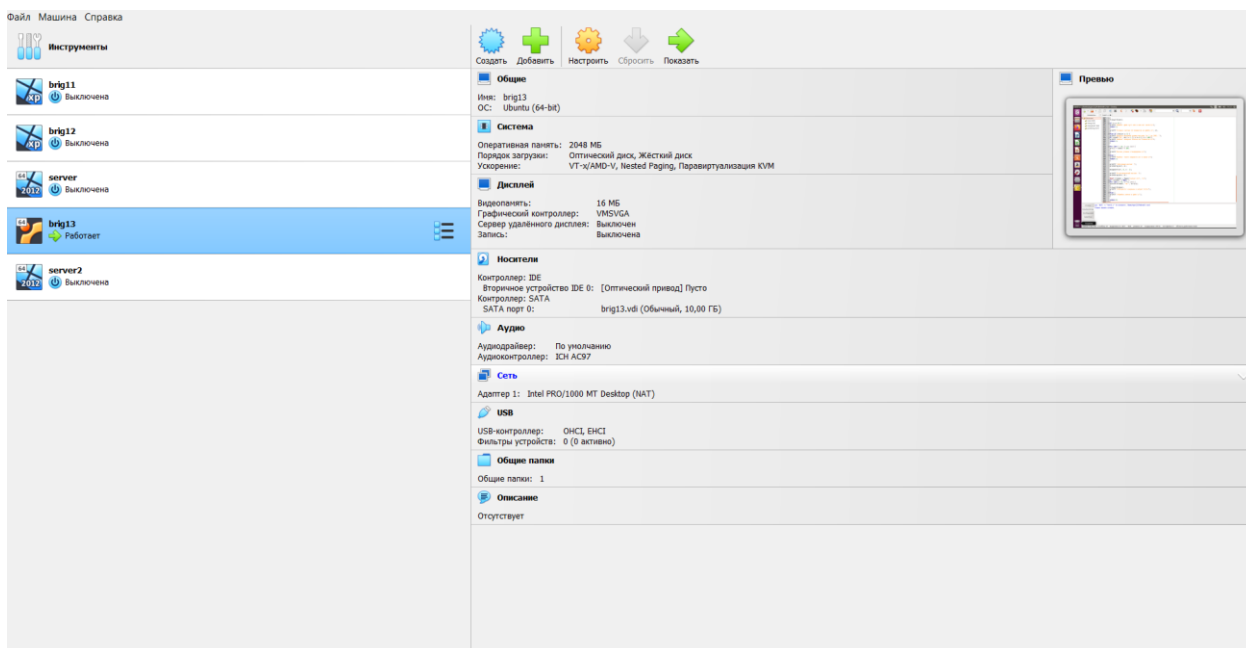


Рисунок 5 – Интерфейс VirtualBox

1.2 Установка виртуальной машины на VirtualBox

Ввели имя виртуальной машины и указали образ ISO. Изображение окна представлено на рисунке 6.

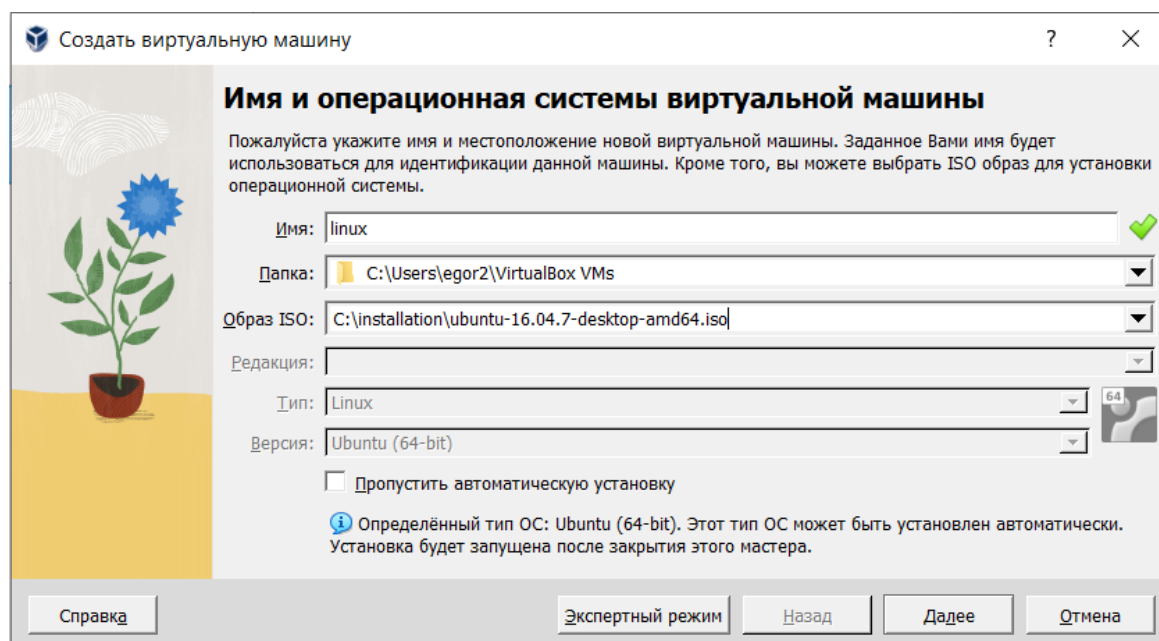


Рисунок 6 – Создание виртуальной машины

Выделили память виртуальной машине. Изображение окна представлено на рисунке 7.

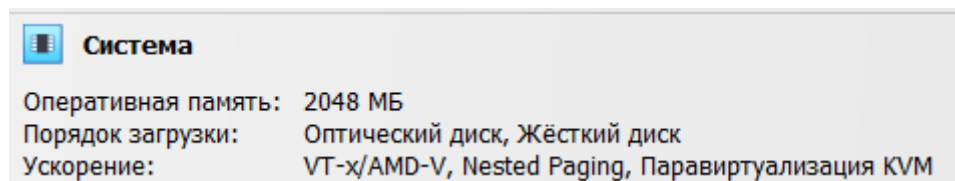


Рисунок 7 – Выделение памяти

Отредактировали оставшиеся параметры. Изображение окна представлено на рисунке 8.

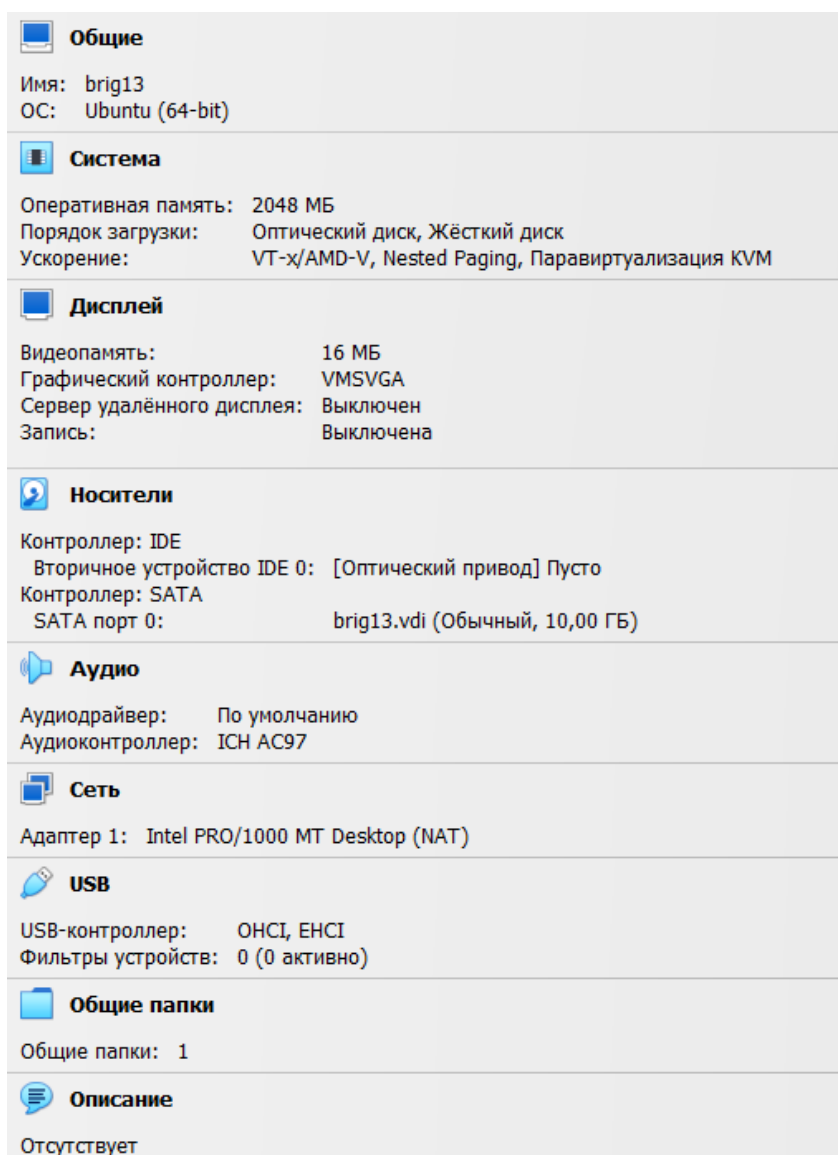


Рисунок 8 – Параметры виртуальной машины

1.3 ОС Linux Ubuntu

В качестве базовой программной среды для выполнения исследовательской части работы был выбран дистрибутив Ubuntu Linux, созданный на основе инфраструктуры Debian GNU/Linux. Данная операционная система занимает лидирующие позиции в академической среде, сфере системного администрирования и промышленной разработки программных продуктов. Популярность дистрибутива обусловлена его высокой отказоустойчивостью, продвинутыми механизмами разграничения прав доступа, эффективным управлением аппаратными ресурсами, а также наличием обширного репозитория специализированного софта для программирования.

Архитектура операционной системы построена по модульному принципу, где каждый программный уровень изолирован и выполняет строго определенный комплекс задач по обеспечению жизнедеятельности системы:

1. Системный загрузчик (GRUB — Grand Unified Bootloader): это первичный программный модуль, который активируется сразу после завершения тестов BIOS/UEFI. Он отвечает за инициализацию оборудования на базовом уровне, выбор доступного ядра операционной системы из конфигурационного списка и последующее конфигурирование параметров его запуска с передачей управления в оперативную память.

2. Ядро Linux: центральный, низкоуровневый компонент ОС, функционирующий в режиме супервизора. Ядро выступает в роли главного абстрактного моста между аппаратной частью ЭВМ (физическая память, процессорное время, контроллеры ввода-вывода) и прикладным программным кодом. Оно осуществляет планирование потоков вычислений, виртуализацию памяти, организацию файловых систем и обработку прерываний от периферийных устройств.

3. Системные службы (демоны): специализированные фоновые процессы, которые инициализируются на ранних этапах загрузки ОС диспетчером системы (например, `systemd`). Демоны функционируют без прямого взаимодействия с интерфейсом пользователя и в непрерывном режиме решают сервисные задачи: управление сетевыми соединениями, логирование системных событий, разграничение межсетевого экрана и контроль планировщика задач.

4. Командная оболочка и утилиты (CLI - Command Line Interface): интерпретатор команд (по умолчанию `bash`), обеспечивающий интерактивное текстовое взаимодействие пользователя с операционной системой. Оболочка позволяет гибко управлять процессами, конфигурировать права доступа к файловым объектам и автоматизировать рутинные операции с помощью скриптов, что является фундаментальным инструментом для любого разработчика.

5. Графическая подсистема и пользовательское окружение: верхний слой архитектуры, включающий в себя дисплейный сервер (Xorg или Wayland) и графическую среду рабочего стола (GNOME). Данный модуль отвечает за отрисовку оконных интерфейсов, обработку событий мыши и клавиатуры, предоставляя пользователю интуитивно понятную визуальную среду для взаимодействия с прикладным ПО.

Открытая модульная структура ОС Ubuntu предоставляет разработчику прозрачный контроль над процессами компиляции кода. Это делает её идеальной средой для написания консольных приложений на языке Си и проведения точного анализа работы алгоритмов сортировки.

1.4 Установка и настройка ОС Linux Ubuntu на VirtualBox

Для установки ОС Linux Ubuntu скачали с официального сайта файл, содержащий дистрибутив операционной системы, и запустили созданную виртуальную машину. Затем выбрали язык ОС. Изображение окна представлено на рисунке 9.

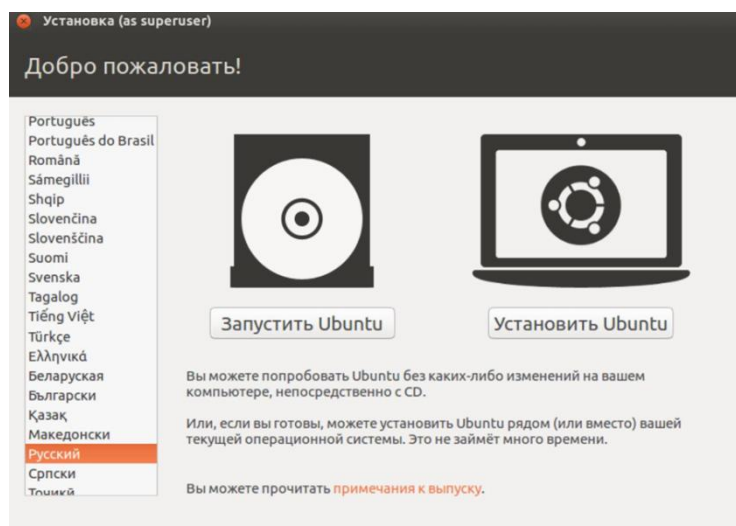


Рисунок 9 – Окно выбора языка и начала установки Ubuntu

Начали подготовку к установке. Изображение окна представлено на рисунке 10.

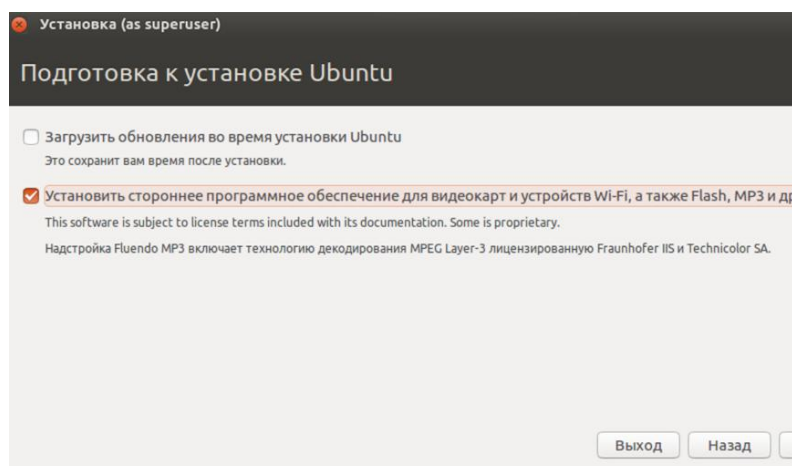


Рисунок 10 – Окно подготовки к установке

Далее выбрали тип установки. Изображение окна представлено на рисунке 11.

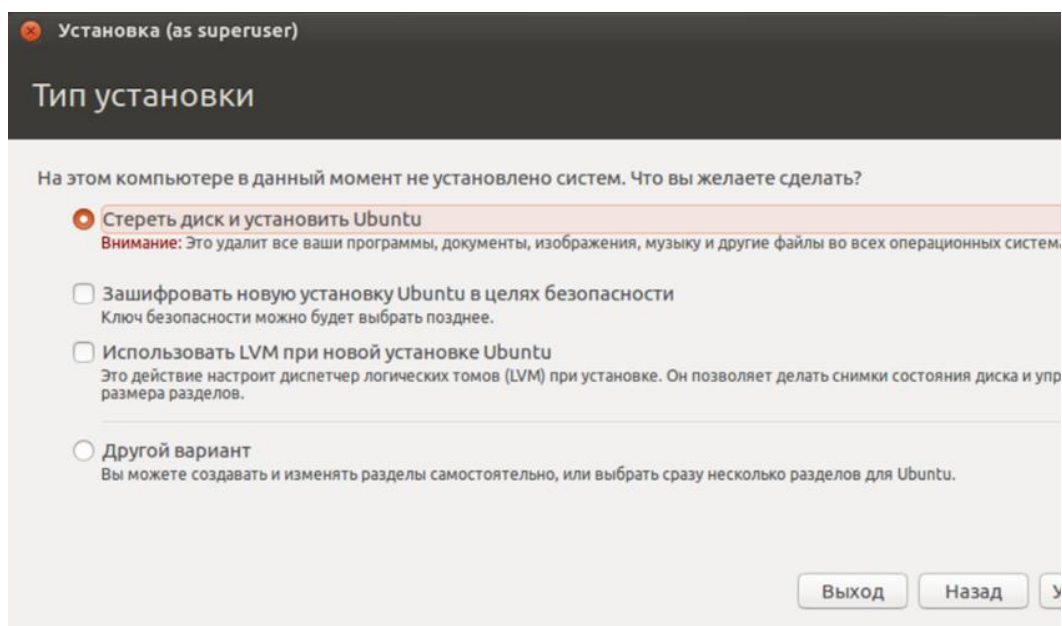


Рисунок 11 – Окно выбора типа установки

Затем был установлен регион. Изображение окна представлено на рисунке 12.

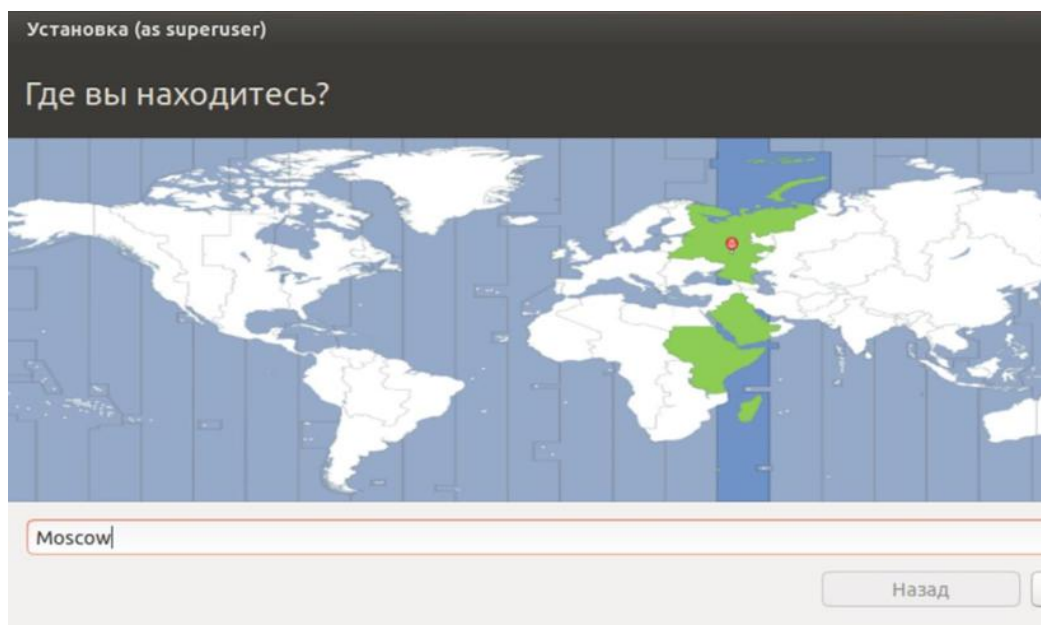


Рисунок 12 – Окно выбора региона

Установили раскладку клавиатуры. Изображение окна представлено на рисунке 13.

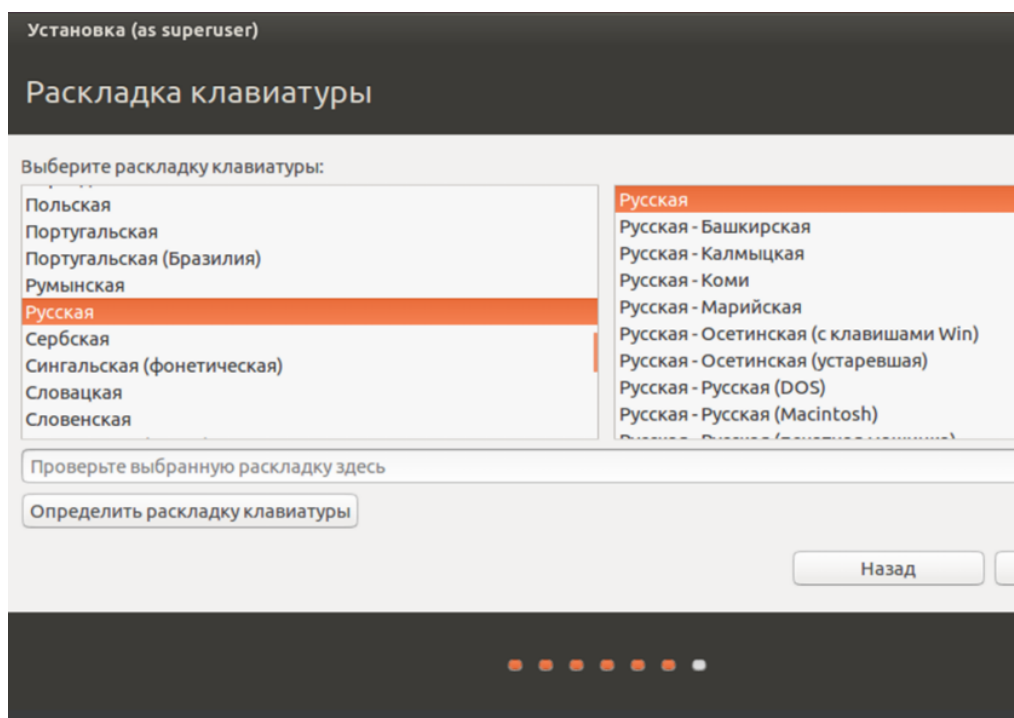


Рисунок 13 – Окно установки раскладки клавиатуры

Успешно завершили установку ОС Linux Ubuntu на VirtualBox.

2. Установка IDE Geany

2.1 Описание IDE Geany

Для написания кода применялась Geany — легковесная интегрированная среда разработки (IDE) на базе библиотеки GTK+, распространяемая по лицензии GNU GPL. Данный инструмент предоставляет всё необходимое для полного цикла программирования на языке Си в едином окне, включая подсветку синтаксиса, встроенный терминал, автозавершение конструкций и удобную навигацию по функциям.

Ключевым фактором при выборе этой среды стали её минимальные системные требования. Поскольку разработка велась внутри виртуальной машины под управлением гостевой ОС Ubuntu, было критически важно использовать нетребовательное к ресурсам ПО. Geany обеспечила высокое быстродействие системы без зависаний.

Дополнительным преимуществом стала автоматическая интеграция среды с системным компилятором GCC (GNU Compiler Collection). Это избавило от необходимости вручную прописывать пути и параметры сборки, позволив сразу приступить к созданию и отладке консольного приложения.

2.2 Описание процесса установки

Зашли в терминал, ввели необходимые команды. Изображение окна терминала представлено на рисунке 14.

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

arcturus@arcturus-VirtualBox:~$ sudo apt update
[sudo] пароль для arcturus:
Сущ:1 http://ru.archive.ubuntu.com/ubuntu xenial InRelease
Сущ:2 http://ru.archive.ubuntu.com/ubuntu xenial-updates InRelease
Сущ:3 http://ru.archive.ubuntu.com/ubuntu xenial-backports InRelease
Сущ:4 http://security.ubuntu.com/ubuntu xenial-security InRelease
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Может быть обновлено 194 пакета. Запустите «apt list --upgradable» для их показа
.
arcturus@arcturus-VirtualBox:~$ sudo apt install geany
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Будут установлены следующие дополнительные пакеты:
  geany-common
Предлагаемые пакеты:
  libvte9
Следующие НОВЫЕ пакеты будут установлены:
  geany geany-common
```

Рисунок 14 – Терминал

Команда `sudo apt update` предназначена для установки обновлений списка пакетов.

Команда `sudo apt install geany` предназначена для установки Geany.

Завершили установку. Затем открыли программу. Изображение интерфейса представлено на рисунке 15.

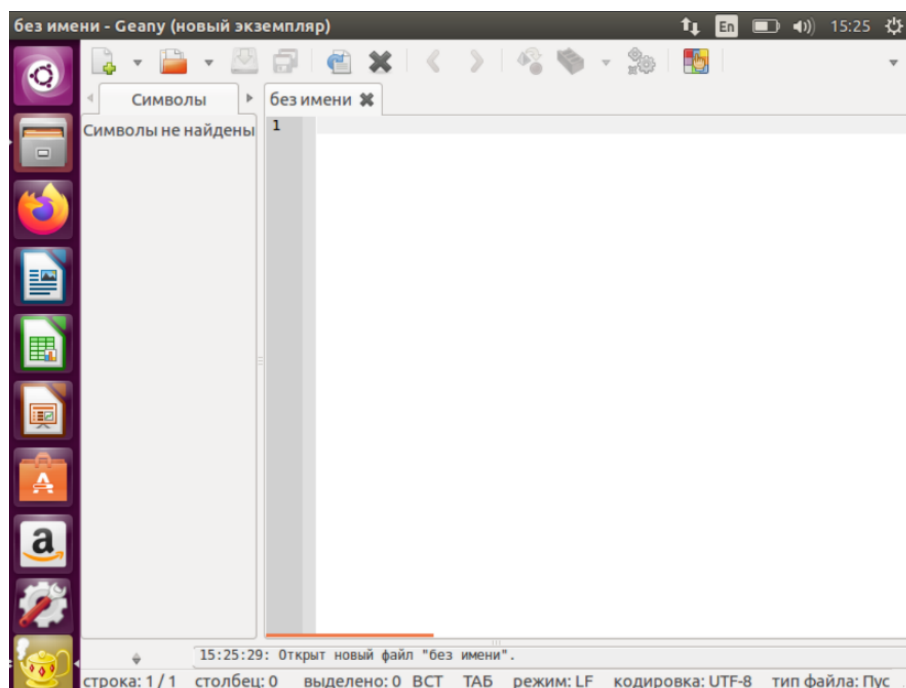


Рисунок 15 – Интерфейс Geany